

AtmoSync

Real-Time Cold Chain Monitoring System

An End-to-End Real-Time IoT Data Engineering Pipeline using Apache Kafka, Snowflake, dbt & Apache Superset

Organization	Infotact Solutions
Project Duration	07 July 2026 – 10 August 2026
Team Members	Purvaja Dodke • Pujari Bhanu Prasad • Satyam Kumar Karn • Shreya Dipak Pawar
Technology Stack	Python • Apache Kafka • Snowflake • dbt • Apache Superset • Docker • SQL • Git & GitHub

CONTENTS

SECTION	PAGE
1. Introduction	2
2. System Architecture	2
3. Project Implementation	2
4. Dashboard & Business Insights	5
5. Version Control & Project Structure	6
6. Project Highlights	6
7. Conclusion	7
8. Future Scope	7

1. Introduction

Project Overview

AtmoSync is an end-to-end real-time cold chain monitoring platform designed to simulate, stream, store, transform, and visualize sensor data from temperature-sensitive shipments. The system combines Python-based IoT simulation with Apache Kafka for event streaming, Snowflake for cloud data storage, dbt for governed transformations, and Apache Superset for operational analytics.

Problem Statement

Cold chain operations depend on maintaining suitable environmental conditions throughout transportation. Delayed detection of temperature excursions, inconsistent sensor data, and fragmented reporting can increase spoilage risk and reduce operational visibility. AtmoSync addresses this by creating a continuous data path from simulated sensors to analytics-ready warehouse models and an interactive monitoring dashboard.

Objectives

- Generate realistic commodity-specific sensor readings for temperature-sensitive shipments.
- Stream readings reliably through Kafka Producer → Broker → Consumer components.
- Persist raw events in Snowflake and create analytics-ready STAGING and MARTS layers with dbt.
- Expose operational KPIs, temperature alerts, container activity, and recent readings through Superset.

Technology Stack

Layer	Technology	Role	Outcome
Simulation	Python	Commodity-aware IoT event generation	Realistic sensor stream
Streaming	Apache Kafka + Docker	Event transport and containerized runtime	Reliable real-time ingestion
Warehouse	Snowflake + SQL	RAW storage and analytics layers	Centralized cloud data
Transform & BI	dbt + Superset	Dimensional modeling and visualization	Analytics-ready insights

2. System Architecture

The pipeline follows a layered architecture. The simulator emits structured events; Kafka transports them in real time; the consumer writes raw records into Snowflake; dbt applies staging and dimensional transformations; and Superset consumes the MARTS layer for monitoring and decision support.

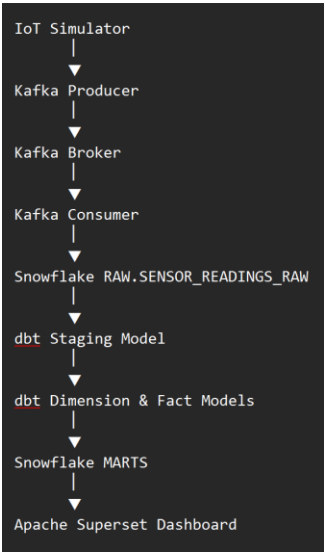


Figure 1. End-to-End Data Engineering Pipeline



Figure 2. dbt Data Lineage

The lineage graph shows the raw sensor and pricing sources feeding the staging model, which then supports DIM_CONTAINER, DIM_ROUTE, and FACT_SENSOR_READINGS. This separates ingestion from business-ready modeling and makes downstream analytics easier to maintain.

3. Project Implementation

Step 1 — Python IoT Simulator

The simulator generates shipment-level events containing Reading_ID, Container_ID, commodity, category, origin, destination, temperature, humidity, vibration, battery level, speed, GPS coordinates, timestamp, shelf life, pricing, optimal storage conditions, and risk classification. Commodity-specific rules make the generated data behave differently for fruits, vegetables, frozen foods, exotic produce, and pharmaceuticals.

Step 2 — Kafka Producer

The Producer serializes each simulated sensor event and publishes it to the Kafka topic. The implementation was refined to keep the stream continuous while preserving realistic values across shipment types.

```
Terminal
/ Sent Sensor Data
{
  "Reading_ID": "R001413",
  "Container_ID": "A103",
  "Commodity": "Banana",
  "Category": "Fruits & Vegetables",
  "Origin": "Bangalore",
  "Destination": "Mumbai",
  "Temperature": 3.3,
  "Humidity": 98,
  "Vibration": 0.19,
  "Battery_Level": 67,
  "Speed": 58.7,
  "Latitude": 12.96929,
  "Longitude": 77.663551,
  "Timestamp": "2026-08-07 21:11:10"
}

/ Sent Sensor Data
{
  "Reading_ID": "R001414",
  "Container_ID": "A111",
  "Commodity": "Medicine",
  "Category": "Pharmaceuticals",
  "Origin": "Pune",
  "Destination": "Lucknow",
  "Temperature": 2.37,
  "Humidity": 62,
  "Vibration": 0.28,
  "Battery_Level": 93,
  "Speed": 79.4,
  "Latitude": 18.50724,
  "Longitude": 73.813475,
  "Timestamp": "2026-08-07 21:11:15"
}

/ Sent Sensor Data
{
  "Reading_ID": "R001415",
  "Container_ID": "A112",
  "Commodity": "Frozen Fish",
  "Category": "Frozen Foods",
  "Origin": "Mumbai",
  "Destination": "Delhi",
  "Temperature": 15.51,
```

Figure 3. Kafka Producer Streaming Sensor Data

Step 3 — Kafka Consumer

The Consumer subscribes to the stream, validates the incoming event payload, and inserts each reading into the Snowflake RAW layer. The terminal output demonstrates successful ingestion and confirms that individual Reading_ID values are persisted as the stream progresses.

```
Terminal
Inserted R001441 into Snowflake
{
  "Reading_ID": "R001441",
  "Container_ID": "A103",
  "Commodity": "Banana",
  "Category": "Fruits",
  "Origin": "Mumbai",
  "Destination": "Jaipur",
  "Temperature": 14.47,
  "Humidity": 98,
  "Vibration": 0.55,
  "Battery_Level": 49,
  "Speed": 41.3,
  "Latitude": 15.084411,
  "Longitude": 72.762574,
  "Timestamp": "2026-08-07 21:32:05"
}

Inserted R001442 into Snowflake
{
  "Reading_ID": "R001443",
  "Container_ID": "A108",
  "Commodity": "Onion",
  "Category": "Vegetables",
  "Origin": "Visakhapatnam",
  "Destination": "Bhubaneswar",
  "Temperature": 0.43,
  "Humidity": 65,
  "Vibration": 0.33,
  "Battery_Level": 92,
  "Speed": 67.4,
  "Latitude": 17.682211,
  "Longitude": 83.249189,
  "Timestamp": "2026-08-07 21:32:10"
}

Inserted R001443 into Snowflake
{
  "Reading_ID": "R001444",
  "Container_ID": "A102",
  "Commodity": "Orange",
  "Category": "Fruits",
  "Origin": "Chennai",
  "Destination": "Coimbatore",
  "Temperature": 7.13,
  "Humidity": 85,
```

Figure 4. Kafka Consumer Receiving Sensor Data

Step 4 — Containerized Runtime

Kafka and ZooKeeper were run through Docker to provide a repeatable local runtime. A startup issue affecting the broker/container configuration was resolved so both services reached a healthy running state before streaming was initiated.

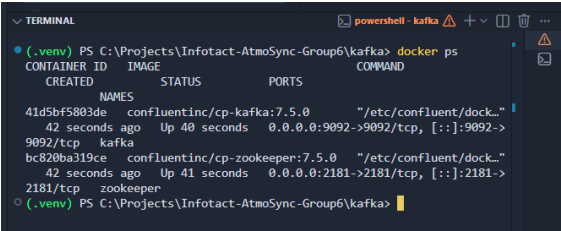


Figure 5. Docker Containers Running Kafka and ZooKeeper

Engineering focus

The pipeline was optimized around reliability and realism: controlled sensor ranges, simulated missing values, commodity-specific storage conditions, risk classification, and reduced unrealistic temperature spikes from 5% to 1%.

Step 5 — Snowflake Data Warehouse

Snowflake provides the persistence and analytical foundation for AtmoSync. The database is organized into RAW, STAGING, and MARTS schemas. RAW retains ingested sensor and commodity pricing data, STAGING provides a cleaned relational view, and MARTS contains the dimensional and fact models consumed by analytics.

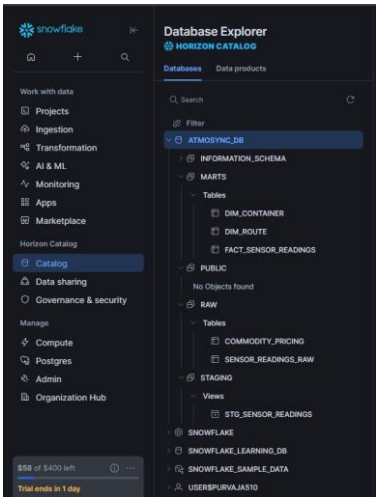


Figure 6. Snowflake Database Structure (RAW, STAGING, MARTS)

A screenshot of a Snowflake query results table. The query is 'SELECT * FROM FACT_SENSOR_READINGS LIMIT 20;'. The table has 20 rows and 15 columns. The columns are: CONTAINER_ID, COMMODITY, CATEGORY, PRICE_PER_KG, SHELF_LIFE_DAYS, OPTIMAL_TEMP_MIN, OPTIMAL_TEMP_MAX, RISK_LEVEL, ROUTE_KEY, SOURCE, DESTINATION, and TEMPERATURE. The data includes various commodities like Potato, Broccoli, Carrot, and Apple, with their respective prices, shelf lives, temperature ranges, risk levels, and routes. The table is displayed in a grid format with alternating row colors.

Figure 7. Final Analytics Table

FACT_SENSOR_READINGS contains the analytical grain for sensor events and is enriched with commodity, category, route, source/destination, temperature, and risk attributes. The model supports operational analysis while preserving the raw event stream separately.

Step 6 — dbt Transformations

dbt operationalizes the transformation layer using SQL models and dependencies. The execution shown below completed four models: one STAGING view and three MARTS tables—DIM_CONTAINER, DIM_ROUTE, and FACT_SENSOR_READINGS—with all configured tests and model execution completing successfully.

```

TERMINAL
(.venv) PS C:\Projects\Infotact-AtmoSync-Group6\atmosync_dbt> dbt run
13:03:16 Running with dbt=1.12.0
13:03:17 Registered adapter: snowflake=1.12.0
13:03:18 Found 4 models, 8 data tests, 1 seed, 2 sources, 546 macros
13:03:18 Concurrency: 4 threads (target='dev')
13:03:18
13:03:21 1 of 4 START sql view model STAGING.stg_sensor_readings .....
..... [RUN]
13:03:22 1 of 4 OK created sql view model STAGING.stg_sensor_readings .....
..... [SUCCESS 1 in 0.78s]
13:03:22 2 of 4 START sql table model MARTS.dim_container .....
..... [RUN]
13:03:22 3 of 4 START sql table model MARTS.dim_route .....
..... [RUN]
13:03:22 4 of 4 START sql table model MARTS.fact_sensor_readings .....
..... [RUN]
13:03:23 2 of 4 OK created sql table model MARTS.dim_container .....
..... [SUCCESS 15 in 1.14s]
13:03:23 4 of 4 OK created sql table model MARTS.fact_sensor_readings .....
..... [SUCCESS 36913 in 1.26s]
13:03:23 3 of 4 OK created sql table model MARTS.dim_route .....
..... [SUCCESS 28 in 1.33s]
13:03:25
13:03:25 Finished running 3 table models, 1 view model in 0 hours 0 mi
nutes and 6.33 seconds (6.33s).
13:03:25
13:03:25 Completed successfully
13:03:25 Done. PASS=4 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=4
(.venv) PS C:\Projects\Infotact-AtmoSync-Group6\atmosync_dbt>

```

Figure 8. Successful dbt Model Execution

4. Dashboard & Business Insights

Apache Superset provides the operational analytics layer. The dashboard combines KPI cards, time-series monitoring, categorical distributions, alert analysis, container activity, and recent sensor records. Interactive filters allow users to narrow the view by risk level, commodity, destination, container, and category.

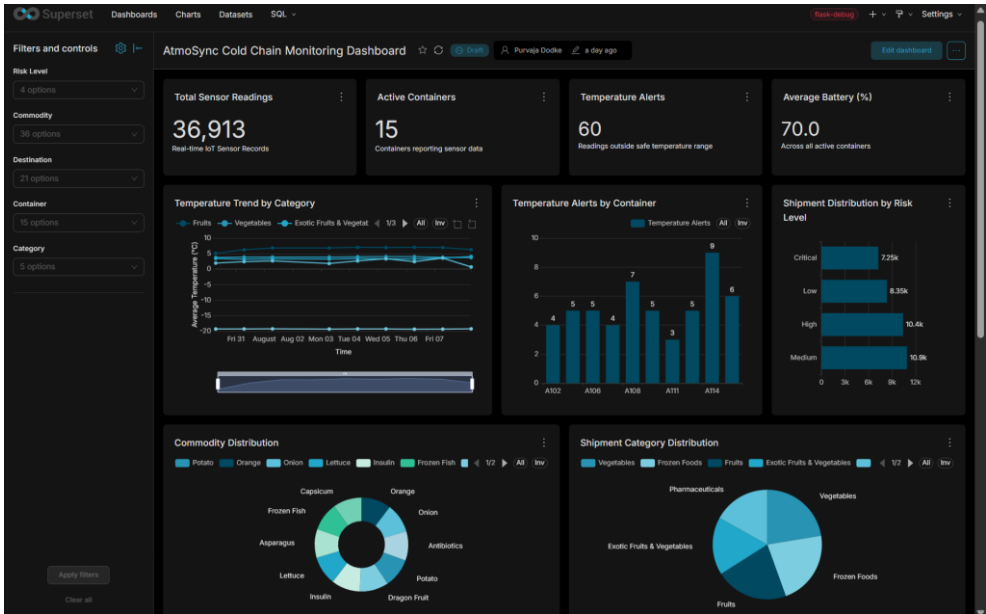


Figure 9. AtmoSync Operations Dashboard

The primary view highlights 36,913 sensor readings, 15 active containers, 60 temperature alerts, and an average battery level of 70%. Temperature trends and alert counts help operators identify excursions, while shipment and commodity distributions provide context for the affected cargo.

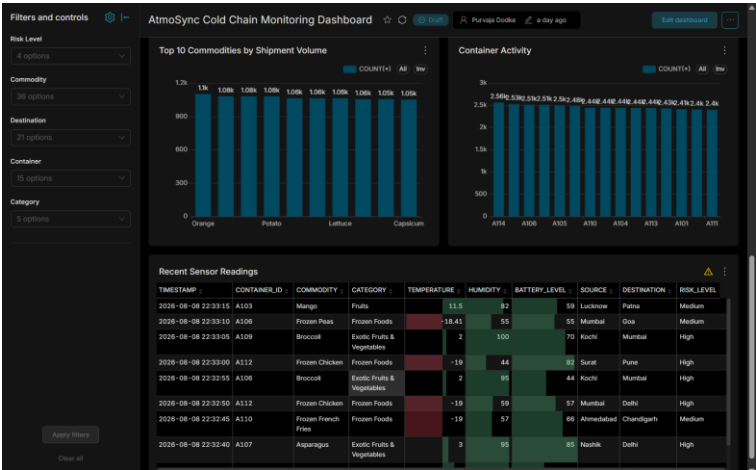


Figure 10. Operational KPIs and Live Sensor Readings

Recent Sensor Readings provide near-real-time visibility into timestamp, container, commodity, category, temperature, humidity, battery, route endpoints, and risk level. This supports rapid investigation of anomalous conditions without leaving the monitoring interface.

5. Version Control & Project Structure

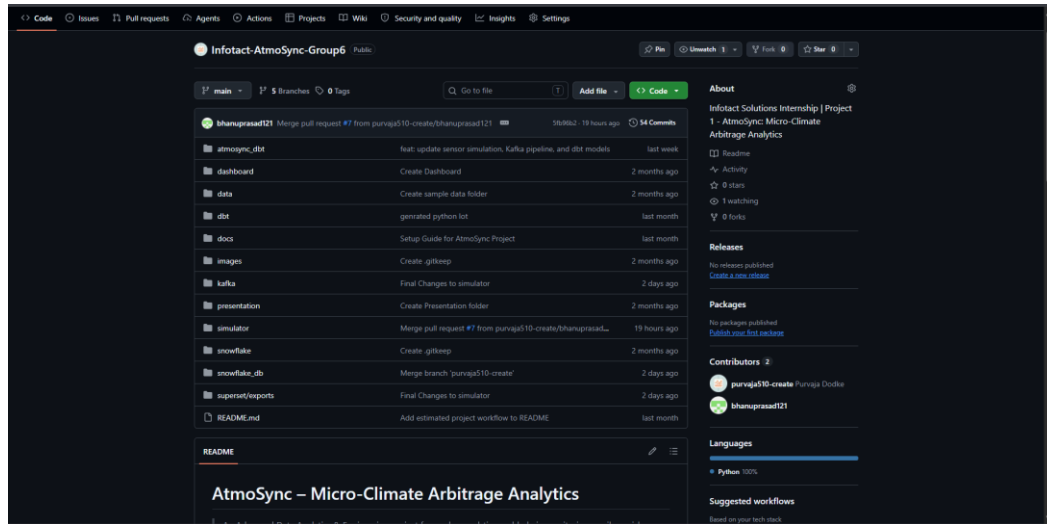


Figure 11. GitHub Repository

GitHub was used for source control, collaboration, and traceability. The repository contains dedicated modules for the simulator, Kafka pipeline, Snowflake assets, dbt project, Superset exports, documentation, data, and presentation materials. Branches and pull requests support coordinated team development.

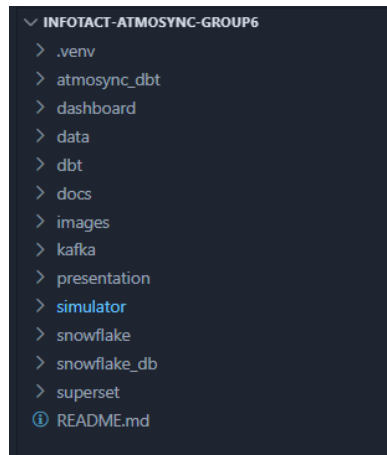


Figure 12. Project Directory Structure

6. Project Highlights

- Real-time IoT simulation with commodity-specific rules and realistic sensor attributes.
- Kafka Producer/Consumer pipeline for continuous event streaming.
- Snowflake RAW → STAGING → MARTS warehouse architecture.
- dbt dimensional modeling using DIM_CONTAINER, DIM_ROUTE, and FACT_SENSOR_READINGS.
- Interactive Apache Superset dashboard with operational KPIs and live-style readings.
- Risk classification, missing-value simulation, and temperature excursion monitoring.
- End-to-end data engineering flow from generation through business visualization.

Challenges & Optimizations

- Kafka startup issue: diagnosed Docker broker/ZooKeeper startup and configuration dependencies.
- Simulator realism: introduced commodity-specific temperature ranges, storage conditions, shelf life, pricing, and risk rules.
- Missing values: simulated incomplete sensor attributes to make downstream handling more representative of production conditions.
- Temperature spikes: reduced the unrealistic spike probability from 5% to 1% to improve signal quality.
- Dimensional modeling: separated reusable dimensions from the sensor fact table to support scalable analytics.
- Dashboard refinement: combined KPIs, alerts, trends, distributions, container activity, and recent readings into a single operational view.

7. Conclusion

End-to-end outcome

AtmoSync demonstrates a complete real-time IoT data engineering workflow in which simulated cold-chain events move through Kafka into Snowflake, are transformed with dbt into governed analytical models, and are surfaced through Apache Superset. The solution converts raw streaming events into operationally meaningful indicators such as temperature alerts, active containers, shipment distributions, battery status, and risk levels.

From an engineering perspective, the project brings together event streaming, cloud warehousing, SQL-based transformation, dimensional modeling, containerized services, and business intelligence in a single integrated pipeline. The implemented controls around commodity-specific behavior, missing-value simulation, risk classification, and temperature-spike optimization improve the realism and analytical usefulness of the simulated environment.

8. Future Scope

Enhancement	Expected Value
Apache Airflow	Orchestrate ingestion, transformation, testing, and scheduled data-quality workflows.
Machine Learning	Detect anomalous sensor behavior and learn dynamic thresholds from historical patterns.
Predictive Spoilage Analytics	Estimate spoilage probability and remaining shelf life using temperature exposure and route history.
Live GPS Tracking	Add map-based shipment tracking and route deviation monitoring.
Email/SMS Alerts	Trigger notifications when temperature, battery, vibration, or risk thresholds are breached.
Mobile Dashboard	Provide a responsive operational interface for field and logistics teams.

— END OF PROJECT DOCUMENTATION —